

PAR3DNSS documentation, Part 1

In this chapter detailed formulation of all the procedures implemented are discussed. We start with the governing equations, followed by finite volume approximation and then linear solver. Immersed boundary technique used is also described thereafter. Understanding of the technical details of the numerical methods adopted will be useful in the next chapter where various inputs are described.

1 Governing equations

The mass and momentum and energy conservation equations for incompressible flow with constant fluid properties are written below.

Continuity equation:

$$\frac{\partial u_i}{\partial x_i} = 0 \quad (1)$$

Momentum equation:

$$\frac{\partial u_i}{\partial t} + \frac{\partial(u_i u_j)}{\partial x_j} = -\frac{\partial p}{\partial x_i} + \alpha_M \frac{\partial^2 u_i}{\partial x_j \partial x_j} + \alpha_B T \delta_{in} + S_a(u_i, p, T) + S_p(x_i, t) \quad (2)$$

Energy equation:

$$\frac{\partial T}{\partial t} + \frac{\partial(u_j T)}{\partial x_j} = \alpha_T \frac{\partial^2 T}{\partial x_j \partial x_j} + S_t(T, x_i, t) \quad (3)$$

Passive or active scalar equation:

$$\frac{\partial \phi}{\partial t} + \frac{\partial(u_j \phi)}{\partial x_j} = \alpha_\phi \frac{\partial^2 \phi}{\partial x_j \partial x_j} + S_\phi \quad (4)$$

For isothermal flows the energy equation is discarded and with an appropriate choice of velocity (U) and length (L) scales $\alpha_M = Re^{-1}$ where Reynolds number $Re = UL/\nu$. On the other hand, flows with temperature variation are categorized in three types: for **forced convection** $\alpha_M = Re^{-1}$, $\alpha_T = (RePr)^{-1}$, for **natural convection** $\alpha_M = (PrRa^{-1})^{1/2}$, $\alpha_T = (PrRa)^{-1/2}$ and $\alpha_B = 1$, for **mixed convection** $\alpha_M = Re^{-1}$, $\alpha_T = (RePr)^{-1}$ and $\alpha_B = RaPr^{-1}Re^{-2}$. Note for the natural convection velocity scale is $(g\beta\Delta TL)^{1/2}$ Rayleigh number $Ra = g\beta\Delta TL^3/\nu\alpha$ and Prandtl number $Pr = \nu/\alpha$. In many situations in order to account for an effect we need to introduce a source term in the momentum equation. Here S_p stands for a **passive source** term which is not a function of dependent variables (u_i, p, T) and S_a refers to **active source** term which is a function of the solution variables (u_i, p, T). The last equation is the convection-diffusion equation for a passive or active scalar with α_ϕ and S_ϕ are diffusion coefficient and the source term. If ϕ in some form appear in the momentum equation it should be treated as an active scalar, else it becomes a passive scalar which does not influence the flow and only “follows” the local velocity.

2 Finite volume formulation

Each term of the continuity equation, Eq. (1), and transport equations, Eqs. (2),(3),(4), are integrated over a hexahedral of volume V shown in Fig. 1. The variables u, v, w and p are defined at the

geometric center of the volume denoted by P . The approximation of the integrals are written below

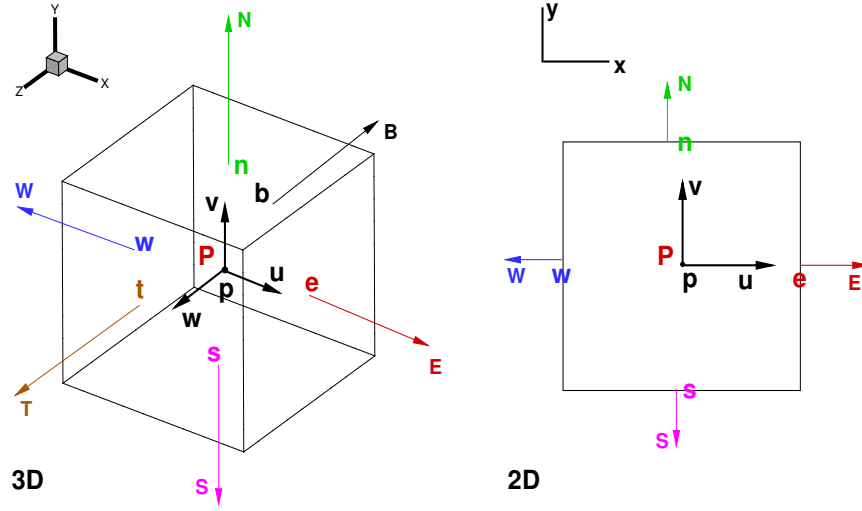


Figure 1: A regular finite volume centered at P showing six faces for 3D and four faces in 2D where all the velocities and the pressure is defined at P .

Continuity equation:

$$\int_V \frac{\partial u_i}{\partial x_i} dV = \int_V \nabla \cdot \mathbf{u} dV = \int_S \mathbf{u} \cdot \hat{\mathbf{n}} dS = \sum_f F_f \quad (5)$$

where F_f denotes mass flux through face f indicated in Fig. 1 and $\hat{\mathbf{n}}$ ($\hat{n}_i$ in index notation) denotes outward normal unit vector of the surface S .

Momentum equation: Integral approximation of different terms in the momentum equation for the velocity component u_i ($i = x, y, z$ i.e., u, v, w) is shown below. Here $n, n + 1$ correspond to n^{th} and $(n + 1)^{th}$ time step, respectively and Δt is the time increment used for the time marching solution. Note i in general corresponds to three linearly independent directions which need not be orthogonal. Such a distinction is felt through the unit outward normals, surface and volume construction.

1. Unsteady term:

$$\int_V u_i dV = \frac{u_{i,P}^{n+1} - u_{i,P}^n}{\Delta t} V_P$$

2. Convective flux:

$$\int_V \frac{\partial(u_i u_j)}{\partial x_j} dV = \int_V \nabla \cdot (u_i u_j) dV = \int_S u_i (u_j \cdot \hat{\mathbf{n}}_j dS) = \int_S u_i dF = \sum_f u_{i,f} F_f$$

3. Diffusion flux: Diffusion flux for the velocity component u_i is shown in detail here (with diffusion coefficient Γ) using Fig. 2 where geometric properties of a face required for calculation of the

diffusion flux are illustrated

$$\int_V \Gamma \frac{\partial^2 u_i}{\partial x_j \partial x_j} dV = \int_V \Gamma \nabla \cdot \nabla u_i dV = \int_S \Gamma \nabla u_i \cdot \hat{\mathbf{n}} dS = \sum_f \Gamma (\nabla u_i \cdot \mathbf{S})_f = \sum_f F_{dfi}$$

Outward normal vector of a face (see Fig. 2) can be written as

$$\mathbf{S}_f = \alpha_1 \hat{\mathbf{n}}_1 + \alpha_2 \hat{\mathbf{n}}_2 + \alpha_3 \hat{\mathbf{n}}_3 \quad (6)$$

where $\hat{\mathbf{n}}_1$, $\hat{\mathbf{n}}_2$, $\hat{\mathbf{n}}_3$ are three linearly independent vectors (not necessarily orthogonal). Thus

$$\nabla u_i \cdot \mathbf{S}_f = \alpha_1 \nabla u_i \cdot \hat{\mathbf{n}}_1 + \alpha_2 \nabla u_i \cdot \hat{\mathbf{n}}_2 + \alpha_3 \nabla u_i \cdot \hat{\mathbf{n}}_3$$

If Δx_1 , Δx_2 and Δx_3 are three line segments in the three directions mentioned in Eq. (6) and Δu_{i1} , Δu_{i2} and Δu_{i3} are differential change in u_i along them, then

$$\Delta u_{i1} = \nabla u_i \cdot \hat{\mathbf{n}}_1 \Delta x_1, \Delta u_{i2} = \nabla u_i \cdot \hat{\mathbf{n}}_2 \Delta x_2, \Delta u_{i3} = \nabla u_i \cdot \hat{\mathbf{n}}_3 \Delta x_3$$

Thus

$$(\nabla u_i \cdot \mathbf{S})_f = \alpha_1 \frac{\Delta u_{i1}}{\Delta x_1} + \alpha_2 \frac{\Delta u_{i2}}{\Delta x_2} + \alpha_3 \frac{\Delta u_{i3}}{\Delta x_3}$$

If $\hat{\mathbf{n}}_1$, $\hat{\mathbf{n}}_2$, $\hat{\mathbf{n}}_3$ have the components $(\hat{n}_{11}, \hat{n}_{12}, \hat{n}_{13})$, $(\hat{n}_{21}, \hat{n}_{22}, \hat{n}_{23})$ and $(\hat{n}_{31}, \hat{n}_{32}, \hat{n}_{33})$, respectively, then Eq. (6) can be written in matrix form

$$\begin{bmatrix} \hat{n}_{11} & \hat{n}_{12} & \hat{n}_{13} \\ \hat{n}_{21} & \hat{n}_{22} & \hat{n}_{23} \\ \hat{n}_{31} & \hat{n}_{32} & \hat{n}_{33} \end{bmatrix}^T \begin{Bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{Bmatrix} = \begin{bmatrix} S_{f1} \\ S_{f2} \\ S_{f3} \end{bmatrix}$$

Using the above matrix equation α_1 , α_2 , α_3 can be solved for each face of a finite volume.

4. Pressure flux:

$$\begin{aligned} \int_V \frac{\partial p}{\partial x_i} dV &= \int_V \hat{n}_i \cdot \frac{\partial p}{\partial x_j} dV = \int_V (\hat{n}_i \cdot \nabla p + p \nabla \cdot \hat{n}_i) dV \\ &= \int_V \nabla \cdot (p \hat{n}_i) dV = \int_S p (\hat{n}_i \cdot \hat{n}_j dS) = \sum_f p_f S_{f,i} \end{aligned}$$

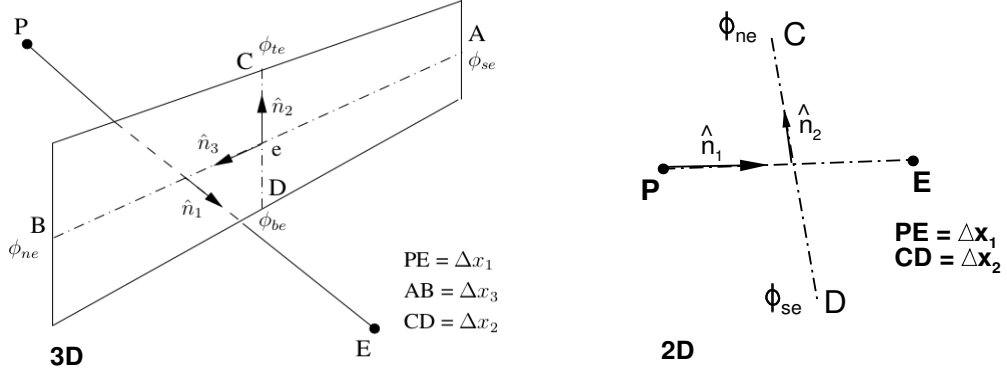


Figure 2: Geometrical features showing edge centers, three unit vectors required for the integral of the Laplacian at the east face.

3 Predictor-corrector method

Using the above finite volume approximations, the continuity and the i^{th} momentum equations can be written in fully discrete form as

$$\sum_f F_f^{n+1} = 0 \quad (7)$$

$$\begin{aligned} \frac{u_{i,P}^{n+1} - u_{i,P}^n}{\Delta t} V_P + \sum_{l=-m}^M \alpha_l \sum_f F_f^{n+l} u_{i,f}^{n+l} = -\frac{1}{2} \left(\sum_f p_f^{n+1} S_{f,i} + \sum_f p_f^n S_{f,i} \right) + \\ \frac{\alpha_M}{2} \left(\sum_f F_{dfi}^{n+1} + \sum_f F_{dfi}^n \right) + \frac{\alpha_B}{2} (T_P^{n+1} + T_P^n) V_P + S_a^{n+1} V_P + S_p^{n+1} V_P \end{aligned} \quad (8)$$

$$\frac{T_P^{n+1} - T_P^n}{\Delta t} V_P + \sum_{l=-m}^M \alpha_l \sum_f F_f^{n+l} T_f^{n+l} = \frac{\alpha_T}{2} \left(\sum_f F_{dfT}^{n+1} + \sum_f F_{dfT}^n \right) + S_t^{n+1} V_P \quad (9)$$

where $M = 1$ refers to **Implicit** time marching with $m = 0, 1, 2, \dots$ refers to $2^{nd}, 3^{rd}, 4^{th}, \dots$ -order Adams-Moulton time integration. On the other hand, $M = 0$ refers to **Explicit** time marching with $m = 1, 2, 3, \dots$ refers to $2^{nd}, 3^{rd}, 4^{th}, \dots$ -order Adams-Bashforth time integration. Note the diffusive, pressure, Buoyancy terms are handled with the 2^{nd} -order Crank-Nicolson scheme in order to retain stability and robustness of the time marching procedure. While the passive source term S_p can be treated implicitly owing to its independence to the solution variables, the active source terms S_a, S_t are treated using appropriate explicit scheme as it depends upon the solution variables.

All the source terms are dropped for the sake of simplicity and to keep the main focus on the issue of velocity-pressure coupling. First a provisional velocity field (u_i^*) is predicted excluding the pressure term and mass flux taken as the latest available one. For the **implicit** method the predictor step is

$$\frac{u_{i,P}^* - u_{i,P}^n}{\Delta t} V_P + \alpha_1 \sum_f F_f^{n+1,q} u_{i,f}^* + \sum_{l=-m}^0 \alpha_l \sum_f F_f^{n+l} u_{i,f}^{n+l} = \frac{\alpha_M}{2} \left(\sum_f F_{dfi}^* + \sum_f F_{dfi}^n \right) \quad (10)$$

where $F_f^{n+1,q}$ is computed from the latest available velocity, and to resolve this non-linearity a special inner-loop indicated by the index q for the flux is carried out. This way a set of simultaneous linear system is set up for each component of velocity. On the other hand, this step for the **explicit** method is

$$\frac{u_{i,P}^* - u_{i,P}^n}{\Delta t} V_P + \sum_{l=-m}^0 \alpha_l \sum_f F_f^{n+l} u_{i,f}^{n+l} = \frac{\alpha_M}{2} \left(\sum_f F_{dfi}^* + \sum_f F_{dfi}^n \right) \quad (11)$$

which is already in a linear form that is set up as a linear system. The face velocities are then calculated by adding a pressure gradient term with the linearly interpolated face values from the provisional velocities

$$u_f = L(u_P^*, u_{nb}^*) - \frac{\Delta t}{\rho} (\nabla p)_f \quad (12)$$

Note that L is the linear interpolation of the cell center velocity u_P and the corresponding neighboring velocity u_{nb} . Thus, Eq. (10) or (11) predicts a velocity field using latest available mass flux and Eq. (12) calculates the cell-face velocities by adding a pressure gradient with the linearly interpolated predicted velocity. This procedure is called **momentum interpolation**, i.e., momentum (velocity with normalized density) is linearly interpolated and pressure gradient is added with it to calculate face velocities which will be used in the mass balance equation. Thus, the mass flux is given by

$$F_f = \rho \mathbf{u}_f \cdot \mathbf{S}_f = \rho \mathbf{u}_f^* \cdot \mathbf{S}_f - \Delta t (\nabla p)_f \cdot \mathbf{S}_f = F_f^* - \Delta t (\nabla p)_f \cdot \mathbf{S}_f \quad (13)$$

where

$$F_f^* = \rho \mathbf{u}_f^* \cdot \mathbf{S}_f \quad (14)$$

Now, if Eq. (13) is inserted into the mass balance expression, Eq. (40), the following equation for the pressure is obtained

$$\sum_f F_f^{n+1,q+1} = \sum_f F_f^* - \Delta t \sum_f (\nabla p)_f \cdot \mathbf{S}_f = 0 \implies \sum_f (\nabla p)_f \cdot \mathbf{S}_f = \frac{1}{\Delta t} \sum_f F_f^* \quad (15)$$

Equation (15) is the counterpart of the Poisson equation for pressure-correction in SIMPLE-like formulation. It should be noted here that solution of Eq. (15) takes the center stage of this entire algorithm as the resulting matrix usually is a poorly conditioned one. The converged pressure field obtained from Eq. (15) is used in Eq. (13) to calculate new estimate of mass flux. Here q denotes counter of the flux loop which is continued until mass flux converges within a time-step, i.e., $(\sum (F_f^{n+1,q+1} - F_f^{n+1,q})^2)^{1/2} < \epsilon$. Note the counter q appears only in the implicit technique. The converged mass flux along with the pressure field that estimate it correspond to the divergence-free velocity field. These quantities are then used to solve Eq. (10) or (11) for the velocity field u_i^{n+1} .

4 Solution Algorithm

The sequential steps that constitute the solution procedure is written below

Implicit technique

- 1. Initialize the variables, $\mathbf{u}$, p and start with mass flux $F_f^{n+1,l} = F_f^n$.
- 2. Solve Eq. (10) for u_i^* .

- 3. Compute F_f^* by Eq. (14) required for the pressure equation.
- 4. Solve Eq. (15) for pressure, p .
- 5. Estimate new mass flux $F_f^{n+1,l+1}$ from Eq. (13).
- 6. Check for flux convergence $\|F_f^{n+1,l+1} - F_f^{n+1,l}\| < \epsilon$, ($\|\phi\|$ implies l_2 norm of ϕ) i.e., if this condition is satisfied go to the next step, else set $l = l + 1$ and repeat steps 2-5 till flux convergence is reached.
- 7. Solve Eq. (41) for the velocity field u_i^{n+1} using converged mass flux and pressure field.
- 8. Solve Eq. (9) for temperature in case of forced convection.
- 9. Set $n = n + 1$ and repeat steps 2-8 for the next time step and thus march in time.

Explicit technique

- 1. Initialize the variables, u , p and start with mass flux F_f^n and F_f^{n-1} .
- 2. Solve Eq. (9) for temperature so that it can be treated implicitly in the momentum equation for free convection.
- 3. Solve Eq. (11) for u_i^* .
- 4. Compute F_f^* by Eq. (14) required for the pressure equation.
- 5. Solve Eq. (15) for pressure, p .
- 6. Estimate new mass flux F_f^{n+1} from Eq. (13).
- 7. Solve Eq. (41) for the velocity field u_i^{n+1} using converged mass flux and pressure field.
- 8. Set $n = n + 1$ and repeat steps 2-7 for the next time step and thus march in time.

Note that for the explicit techniques flux loop is not required as mass flux in the convective term is treated explicitly.

5 Iterative solution of system of linear algebraic equations

System of linear equations arising from Eq. (10) or (11), (15), (41), (9) can be cast into the form

$$A\phi = B \quad (16)$$

Residual form

Iterative methods, in general, are of the form

$$M \phi^{k+1} = N \phi^k + B \quad (17)$$

where M , N are iteration matrices and k is the iteration counter. At the convergence as $\phi^{k+1} \rightarrow \phi^k \rightarrow \phi$, we must have

$$A = M - N \quad (18)$$

Note that Eq. (17) can be cast into the residual form in the following way

$$M \phi^{k+1} - M \phi^k = B - (M - N) \phi^k \implies M \delta^{k+1} = \rho^k \quad (19)$$

where $\delta^{k+1} = \phi^{k+1} - \phi^k$ and $\rho^k = B - (M - N) \phi^k = B - A \phi^k$ are the updates and residual at $k + 1$ iteration level, respectively.

Relaxation

Note ϕ^{k+1} can be found by first solving $\delta^{k+1} = M^{-1} \rho^k$ followed by the update $\phi^{k+1} = \phi^k + \delta^{k+1}$. Clearly choice of M is driven by this step and a suitable form of M is required for a cheap inversion. Moreover, as M approaches A , Eqs. (16) and (17) become equivalent and expectedly less efforts go in iterations. Thus, choice of M should be such that it is easily invertible and is close to A . Now, additive splitting of A can be written as

$$A = L + D + U$$

where L, U, D are the lower, upper and diagonal parts of A . For $M = D$ inversion of M is the cheapest but a poor approximation of M leads to a method, known as **Jacobi iteration**, converges very slow which is often termed as “diagonal scaling”. If M is taken as $L + D$ the choice is better but not the best though again inversion is cheap, inverse of a triangular matrix remains triangular, $\delta^{k+1} = (L + D)^{-1} \rho^k$. This method is known as the famous **Gauss-Seidel** technique. A faster variant of this method is the popular **SOR** technique.

Successive over-relaxation (SOR) technique

In the SOR technique, the iteration matrix M (see Eq. (17)) is chosen as the lower triangular part of A , i.e., $L + D$. A generic algebraic equation that results from approximation of the integrals at a finite volume having cell center P and neighbors E, W, N, S, T, B is

$$a_E \phi_E^k + a_N \phi_N^k + a_T \phi_T^k + a_P \phi_P^{k+1} + a_W \phi_W^{k+1} + a_S \phi_S^{k+1} + a_B \phi_B^{k+1} = b_P \quad (20)$$

Here iterative counter k and $k + 1$ correspond to the latest values available at a point, and we seek an updating formula for ϕ_P^{k+1} . The above equation can be written as

$$\begin{aligned} a_P \phi_P^{k+1} &= b_P - (a_E \phi_E^k + a_N \phi_N^k + a_T \phi_T^k + a_W \phi_W^{k+1} + a_S \phi_S^{k+1} + a_B \phi_B^{k+1}) \\ a_P \phi_P^{k+1} &= a_P \phi_P^k + b_P - (a_E \phi_E^k + a_N \phi_N^k + a_T \phi_T^k + a_P \phi_P^k + a_W \phi_W^{k+1} + a_S \phi_S^{k+1} + a_B \phi_B^{k+1}) \\ a_P \phi_P^{k+1} &= a_P \phi_P^k + [b_P - (\sum_i a_i \phi_i)^{\text{latest}}] \\ a_P \phi_P^{k+1} &= a_P \phi_P^k + \text{Residual}_P \\ \phi_P^{k+1} &= \phi_P^k + \frac{\text{Residual}_P}{a_P} \end{aligned} \quad (21)$$

Note that $(\sum_i a_i \phi_i)^{\text{latest}}$ is the latest estimate of the left hand side of Eq. (20) and $\text{Residual}_P = b_P - (\sum_i a_i \phi_i)^{\text{latest}}$ is the residual of Eq. (20) which becomes zero (theoretically) at the convergence. In other words, in Eq. (19) change in ϕ_P^k to ϕ_P^{k+1} becomes zero as Residual_P reduces to zero. Using an over-relaxation factor (ω) the above formula can be written as

$$\phi_P^{k+1} = \phi_P^k + \omega \frac{\text{Residual}_P}{a_P} \quad (22)$$

Multiplicative splitting

One of the better way of splitting A is the LU factorization. However, complete or direct factorization has two main drawbacks. Firstly, level of additional calculations can be quite high if size of A is large. In addition to this, though A is sparse, one can not ensure the same for L and U which spoils the basic structure of A which should be exploited for a faster technique. In view of this, L and U are actually chosen from A though subsequent multiplication LU does not give back A and this is the reason this class of methods are known as **incomplete LU factorization**, in short **ILU**.

Stone's strongly implicit procedure¹ (SIP) is a variant of ILU factorization originally proposed for a pentadiagonal sparse matrix obtained in two-dimensional Cartesian grid. For three-dimensional flows in Cartesian grids numerical approximations result in algebraic equation with seven point stencil (refer to Fig. 3).

$$a_S \phi_S + a_B \phi_B + a_W \phi_W + a_P \phi_P + a_E \phi_E + a_T \phi_T + a_N \phi_N = b \implies A\phi = B \quad (23)$$

In the SIP method the original matrix is LU factorized. In doing so, a seven diagonal matrix can be factorized into the following L and U factors.

$$A = [L_P \ L_W \ L_S \ L_B] [I \ U_E \ U_T \ U_N] \quad (24)$$

where the diagonal of U is chosen as the unit I to make the factorization unique. Note that the L factors are always the diagonals corresponding to locations which is already traversed during the iteration sweep. Thus, the resulting equation becomes

$$\begin{aligned} [L_P \ L_W \ L_S \ L_B] [I \ U_E \ U_T \ U_N] \phi &= B \\ \implies [L_P \ L_W \ L_S \ L_B] [\phi_P \ U_E \phi_E \ U_T \phi_T \ U_N \phi_N] &= B \end{aligned}$$

¹H. L. Stone, Iterative Solution of Implicit Approximation of Multidimensional Partial Differential Equations, SIAM J. Numer. Anal., **5**, 530-558, 1968.

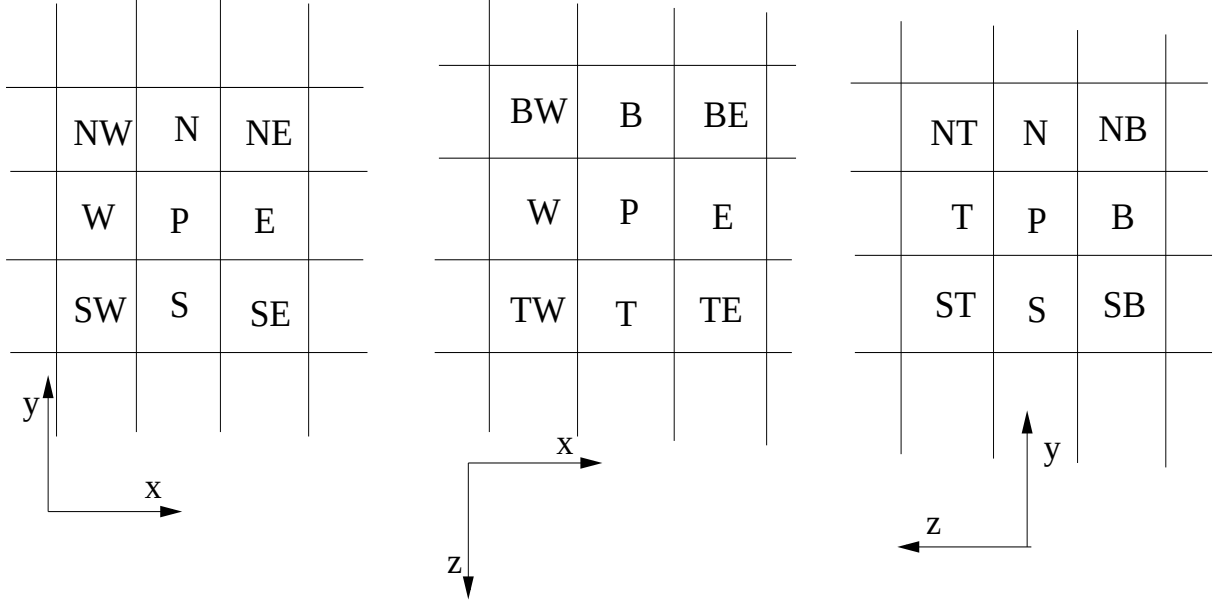


Figure 3: Planar view of finite volumes and their neighbors used in the algebraic equations

The above matrix equation when written for the P node, results in

$$\begin{aligned}
 & (L_P\phi_P + L_P U_E \phi_E + L_P U_T \phi_T + L_P U_N \phi_N) + \\
 & (L_W \phi_W + L_W U_{Ei-1} \phi_P + \boxed{L_W U_{Ti-1} \phi_{TW}} + \boxed{L_W U_{Ni-1} \phi_{NW}}) + \\
 & (L_S \phi_S + \boxed{L_S U_{Ej-1} \phi_{SE}} + \boxed{L_S U_{Tj-1} \phi_{ST}} + L_S U_{Nj-1} \phi_P) + \\
 & (L_B \phi_B + \boxed{L_B U_{Ek-1} \phi_{BE}} + L_B U_{Tk-1} \phi_P + \boxed{L_B U_{Nk-1} \phi_{NB}}) = b
 \end{aligned} \tag{25}$$

where i, j, k directions are shown in Fig. 3, and only the locations which differ from i, j or k are mentioned. For instance, U_{Ei-1} is $U_{Ei-1,j,k}$, U_{Tj-1} is $U_{Ti,j-1,k}$, U_{Nk-1} is $U_{Ni,j,k-1}$ and L_W is $L_{Wi,j,k}$. Note in the above equation, the terms corresponding to the extra diagonals that arise because of LU multiplication are highlighted. Appearance of these extra terms can be immediately noticed by comparing Eqs. (23) and (25). As the iteration matrix M is to be factorized

$$M = LU \tag{26}$$

the diagonals of M can be written from Eq. (25)

$$\begin{aligned}
 M_S &= L_S, \quad M_W = L_W, \quad M_B = L_B, \quad M_N = L_P U_N, \quad M_T = L_P U_T, \quad M_E = L_P U_E \\
 M_P &= L_P + L_W U_{Ei-1} + L_S U_{Nj-1} + L_B U_{Tk-1} \\
 M_{NW} &= L_W U_{Ni-1}, \quad M_{SE} = L_S U_{Ej-1}, \quad M_{TW} = L_W U_{Ti-1} \\
 M_{ST} &= L_S U_{Tj-1}, \quad M_{BE} = L_B U_{Ek-1}, \quad M_{NB} = L_B U_{Nk-1}
 \end{aligned} \tag{27}$$

As the iteration matrix M is given by

$$M = A + N \tag{28}$$

the diagonals of M or the product LU have to be distributed keeping in mind N should be as small as possible for faster convergence. In the standard Incomplete LU (ILU) decomposition N carries

only the extra diagonals (highlighted in Eq. (25)). However, in the SIP method, N is assumed to have non-zero diagonals corresponding to the non-zero locations of M ; i.e., in addition to extra diagonals of M (M_{NW} , M_{SE} , M_{TW} , M_{ST} , M_{BE} and M_{NB}), N also has non-zero diagonals N_S , N_W , N_B , N_P , N_T , N_E , N_N . Thus, in view of faster convergence

$$N \phi \approx 0 \implies N_S \phi_S + N_B \phi_B + N_W \phi_W + N_P \phi_P + N_E \phi_E + N_T \phi_T + N_N \phi_N + M_{NW} \phi_{NW} + M_{SE} \phi_{SE} + M_{TW} \phi_{TW} + M_{ST} \phi_{ST} + M_{BE} \phi_{BE} + M_{NB} \phi_{NB} \approx 0 \quad (29)$$

which implies effect of extra diagonals is nullified by the non-zero diagonals of N . This can be written in the equivalent form as

$$M_{NW}(\phi_{NW} - \phi_{NW}^*) + M_{SE}(\phi_{SE} - \phi_{SE}^*) + M_{TW}(\phi_{TW} - \phi_{TW}^*) + M_{ST}(\phi_{ST} - \phi_{ST}^*) + M_{BE}(\phi_{BE} - \phi_{BE}^*) + M_{NB}(\phi_{NB} - \phi_{NB}^*) \approx 0 \quad (30)$$

where the ϕ^* quantities can be expressed implicitly in terms of neighboring nodal values as

$$\begin{aligned} \phi_{NW}^* &\approx \alpha(\phi_N + \phi_W - \phi_P), \quad \phi_{SE}^* \approx \alpha(\phi_S + \phi_E - \phi_P), \\ \phi_{TW}^* &\approx \alpha(\phi_T + \phi_W - \phi_P), \quad \phi_{ST}^* \approx \alpha(\phi_S + \phi_T - \phi_P), \\ \phi_{BE}^* &\approx \alpha(\phi_B + \phi_E - \phi_P), \quad \phi_{NB}^* \approx \alpha(\phi_N + \phi_B - \phi_P) \end{aligned} \quad (31)$$

where α implicitly relates the nodal values to the locations corresponding to the extra diagonals of M . It has been seen from numerical exercises that $\alpha \approx 0.9$ gives the best convergence, though there is no universal value of it which ensures the best convergence for a range of grid structures having different aspect ratios. When the implicit relations of Eq. (31) are inserted in Eq. (30), the resulting equations is

$$\begin{aligned} M_{NW}(\phi_{NW} - \alpha\phi_N - \alpha\phi_W + \alpha\phi_P) + M_{SE}(\phi_{SE} - \alpha\phi_S - \alpha\phi_E + \alpha\phi_P) + \\ M_{TW}(\phi_{TW} - \alpha\phi_T - \alpha\phi_W + \alpha\phi_P) + M_{ST}(\phi_{ST} - \alpha\phi_S - \alpha\phi_T + \alpha\phi_P) + \\ M_{BE}(\phi_{BE} - \alpha\phi_B - \alpha\phi_E + \alpha\phi_P) + M_{NB}(\phi_{NB} - \alpha\phi_N - \alpha\phi_B + \alpha\phi_P) \approx 0 \end{aligned} \quad (32)$$

Thus, by comparing Eqs. (29) and (32), the non-zero diagonals of N can be obtained as

$$\begin{aligned} N_S &= -\alpha(M_{ST} + M_{SE}) \\ N_B &= -\alpha(M_{BE} + M_{NB}) \\ N_W &= -\alpha(M_{NW} + M_{TW}) \\ N_P &= \alpha(M_{NW} + M_{ST} + M_{SE} + M_{BE} + M_{TW} + M_{NB}) \\ N_E &= -\alpha(M_{SE} + M_{BE}) \\ N_T &= -\alpha(M_{ST} + M_{TW}) \\ M_N &= -\alpha(M_{NW} + M_{NB}) \end{aligned} \quad (33)$$

The matrix relation

$$A + N = M$$

can now be written in terms of their individual diagonals, taking diagonals of N and M from Eqs.

(33) and (27), respectively

$$\begin{aligned}
a_S - \alpha L_S U_{Tj-1} - \alpha L_S U_{Ej-1} &= L_S \\
a_B - \alpha L_B U_{Ek-1} - \alpha L_B U_{Nk-1} &= L_B \\
a_W - \alpha L_W U_{Ni-1} - \alpha L_W U_{Ti-1} &= L_W \\
a_P + \alpha L_W U_{Ni-1} + \alpha L_S U_{Tj-1} + \alpha L_S U_{Ej-1} + \alpha L_B U_{Ek-1} + \\
&\quad \alpha L_W U_{Ti-1} + \alpha L_B U_{Nk-1} = L_P + L_W U_{Ei-1} + L_S U_{Nj-1} + L_B U_{Tk-1} \\
a_E - \alpha L_S U_{Ej-1} - \alpha L_B U_{Ek-1} &= L_P U_E \\
a_T - \alpha L_S U_{Tj-1} - \alpha L_W U_{Ti-1} &= L_P U_T \\
a_N - \alpha L_W U_{Ni-1} - \alpha L_B U_{Nk-1} &= L_P U_N
\end{aligned} \tag{34}$$

By solving seven relations written in Eq. (34) for seven diagonals (L_S , L_B , L_W , L_P , U_E , U_T and U_N), they can be computed in the order given below

$$\begin{aligned}
L_S &= \frac{a_S}{1 + \alpha(U_{Tj-1} + U_{Ej-1})} \\
L_B &= \frac{a_B}{1 + \alpha(U_{Ek-1} + U_{Nk-1})} \\
L_W &= \frac{a_W}{1 + \alpha(U_{Ni-1} + U_{Ti-1})} \\
L_P &= a_P + \alpha(L_W U_{Ni-1} + L_W U_{Ti-1} + L_S U_{Tj-1} + L_S U_{Ej-1} + L_B U_{Ek-1} + L_B U_{Nk-1}) \\
&\quad - L_W U_{Ei-1} - L_S U_{Nj-1} - L_B U_{Tk-1} \\
U_E &= \frac{a_E - \alpha(L_S U_{Ej-1} + L_B U_{Ek-1})}{L_P} \\
U_T &= \frac{a_T - \alpha(L_S U_{Tj-1} + L_W U_{Ti-1})}{L_P} \\
U_N &= \frac{a_N - \alpha(L_W U_{Ni-1} + L_B U_{Nk-1})}{L_P}
\end{aligned} \tag{35}$$

while the MSI diagonals are

$$\begin{aligned}
L_{SW} &= a_{SW} \\
L_S &= \frac{a_S - L_{SW}U_{Ei-1,j-1} - \alpha a_{SE}U_{Ei+1,j-1}}{1 - \alpha U_{Ej-1}U_{Ei+1,j-1}} \\
L_{SE} &= a_{SE} - L_S U_{Ej-1} \\
L_W &= \frac{a_W - L_{SW}U_{Ni-1,j-1} - L_S U_{NWj-1} - 2\alpha L_{SW}U_{NW_{i-1},j-1}}{1 + 2\alpha U_{NW_{i-1}}} \\
L_B &= \frac{a_B}{1 + \alpha(U_{Ek-1} + U_{Nk-1})} \\
L_W &= \frac{a_W}{1 + \alpha(U_{Ni-1} + U_{Ti-1})} \\
L_P &= a_P + \alpha(L_W U_{Ni-1} + L_W U_{Ti-1} + L_S U_{Tj-1} + L_S U_{Ej-1} + L_B U_{Ek-1} + L_B U_{Nk-1}) \\
&\quad - L_W U_{Ei-1} - L_S U_{Nj-1} - L_B U_{Tk-1} \\
U_E &= \frac{a_E - \alpha(L_S U_{Ej-1} + L_B U_{Ek-1})}{L_P} \\
U_T &= \frac{a_T - \alpha(L_S U_{Tj-1} + L_W U_{Ti-1})}{L_P} \\
U_N &= \frac{a_N - \alpha(L_W U_{Ni-1} + L_B U_{Nk-1})}{L_P}
\end{aligned} \tag{36}$$

After obtaining the L and U factors the residual form, Eq. (19), can be solved in two steps

$$\begin{aligned}
&\implies M\delta^{k+1} = \rho^k \\
&\implies LU\delta^{k+1} = \rho^k \\
&\implies \text{step 1: forward substitution: } Ls = \rho^k \\
&\implies \text{step 2: backward substitution: } U\delta^{k+1} = s
\end{aligned} \tag{37}$$

Pre-conditioned Bi-Conjugate Gradient Stabilized method

Among many variants of the BiCGSTAB technique, the algorithm proposed by Zhang² is described here. As this technique does not ensure a monotonic convergence, use of preconditioners not only ensure more smother convergence behavior, but also accelerate the convergence dramatically. SIP splitting is used to obtain the preconditioner. The preconditioned system is $M^{-1}A\phi = M^{-1}\mathbf{b}$, i.e., $\bar{A}\phi = \bar{\mathbf{b}}$. Thus, a matrix-vector multiplication $A\mathbf{x}$ should be replaced by $M^{-1}A\mathbf{x}$ which can be achieved as

$$\mathbf{x}_1 = M^{-1}A\mathbf{x} \implies M\mathbf{x}_1 = A\mathbf{x} \implies Ls = A\mathbf{x}, U\mathbf{x}_1 = s \text{ and finally } A\mathbf{x} = \mathbf{x}_1$$

The optional preconditioner steps are written in red color.

²Shao-Liang Zhang, GPBi-CG: Generalized product-type methods based on Bi-CG for solving non-symmetric linear systems, SIAM J. Sci. Comput., **18**(2), 537-551, 1997.

1. Initialize: $\phi = \phi_{in}$, $\mathbf{r} = \mathbf{b} - A\phi_{in}$, $M\mathbf{r}_1 = \mathbf{r}$, $\mathbf{r} = \mathbf{r}_1$, $\mathbf{r}_0 = \mathbf{r}$, ρ_0 , α , ξ any large value, $k = 0$
2. Compute $M\mathbf{p}_1 = A\mathbf{p}$, $A\mathbf{p} = \mathbf{p}_1$, $\mathbf{p} = \mathbf{r} + \beta(\mathbf{p} - \xi A\mathbf{p})$
3. $\alpha = \frac{\|\mathbf{r} \cdot \mathbf{r}_0\|}{\|A\mathbf{p} \cdot \mathbf{r}_0\|}$
4. Compute $\mathbf{z} = \mathbf{r} - \alpha A\mathbf{p}$
5. $M\mathbf{z}_1 = A\mathbf{z}$, $A\mathbf{z} = \mathbf{z}_1$, $\xi = \frac{\|A\mathbf{z} \cdot \mathbf{z}\|}{\|A\mathbf{z} \cdot A\mathbf{z}\|}$
6. $\phi = \phi + \alpha\mathbf{p} + \xi\mathbf{z}$
 $\mathbf{r} = \mathbf{z} - \xi A\mathbf{z}$
 $\beta = \frac{\alpha\|\mathbf{r}_0 \cdot \mathbf{r}\|}{\xi\|\mathbf{r}_0 \cdot \mathbf{r}\|_{k-1}}$
7. If $\|\mathbf{b} - A\phi\|_{l_2} < \epsilon$, exit;
 else, $k \rightarrow k + 1$ and return to step 2 and continue.

6 Surface area and volume calculation

Figure 4 shows face centers (e, w, n, s, t, b), edge centers ($ne, se, te, be, tn, bn, nw, sw, tw, bw, ts, bs$), neighboring cell centers (E, W, N, S, T, B), and face normal unit vectors ($\hat{n}_e, \hat{n}_w, \hat{n}_n, \hat{n}_s, \hat{n}_t, \hat{n}_b$) for a finite volume. By defining face diagonal vector as $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$ where $\mathbf{r}_i$ and $\mathbf{r}_j$ are the position vectors of the corners i and j , respectively, surface areas are written below

$$\begin{aligned}
 \text{East face : } \mathbf{S}_e &= \frac{1}{2}(\mathbf{r}_{74} \times \mathbf{r}_{83}) \\
 \text{West face : } \mathbf{S}_w &= \frac{1}{2}(\mathbf{r}_{16} \times \mathbf{r}_{52}) \\
 \text{North face : } \mathbf{S}_n &= \frac{1}{2}(\mathbf{r}_{18} \times \mathbf{r}_{45}) \\
 \text{South face : } \mathbf{S}_s &= \frac{1}{2}(\mathbf{r}_{27} \times \mathbf{r}_{63}) \\
 \text{Top face : } \mathbf{S}_t &= \frac{1}{2}(\mathbf{r}_{57} \times \mathbf{r}_{86}) \\
 \text{Bottom face : } \mathbf{S}_b &= \frac{1}{2}(\mathbf{r}_{13} \times \mathbf{r}_{24})
 \end{aligned}$$

The volume is calculated as

$$V = \mathbf{r}_{71} \cdot (\mathbf{S}_n + \mathbf{S}_b + \mathbf{S}_w)$$

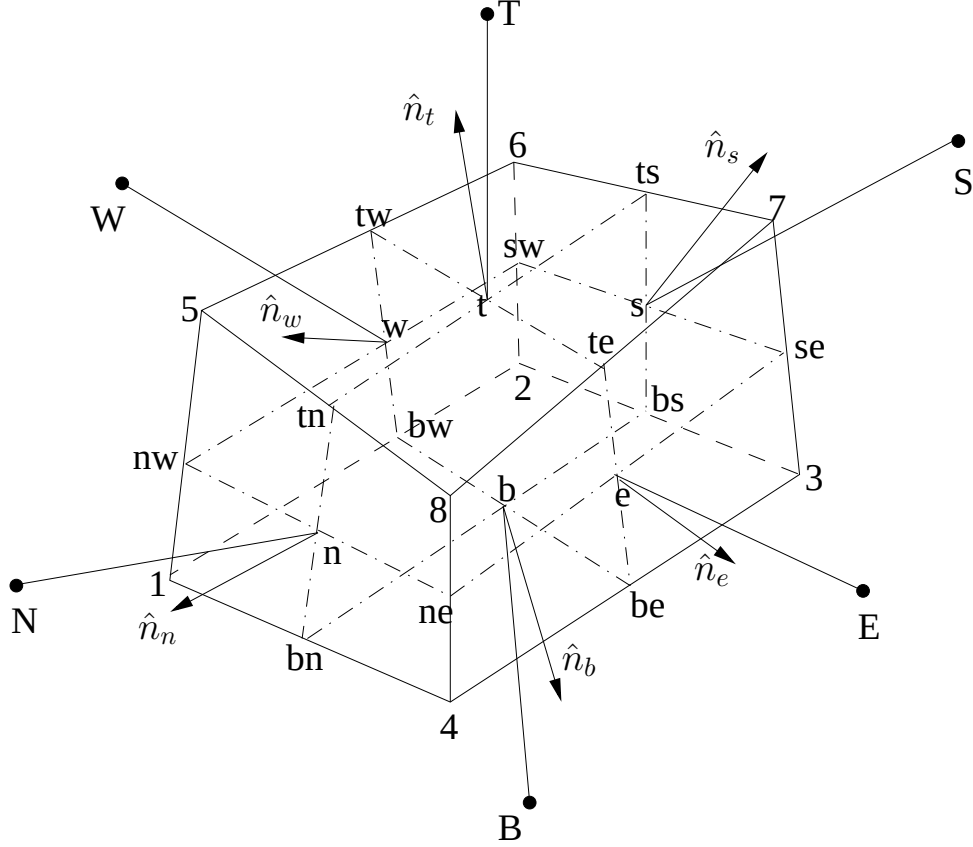


Figure 4: Geometrical features of a typical finite volume

7 Convection schemes

Application of convection scheme is shown here with the help of Fig. 5. The convective flux of a variable ϕ at face f is given by $F_f \phi_f$. As ϕ is defined at the cell centers, an interpolation scheme used for the calculation of the face values determines the accuracy of the convective fluxes being computed.

Central Difference: Here two neighboring cell center values are volume-interpolated to compute the face center value. In Fig. 5, interpolations at the east and west faces are

$$(F_e \phi_e)_{CDS} = F_e \frac{\phi_P V_E + \phi_E V_P}{V_P + V_E}, \quad (F_w \phi_w)_{CDS} = F_w \frac{\phi_P V_W + \phi_W V_P}{V_P + V_W}$$

Note that for rectangular uniform grids the above formula is 2^{nd} -order accurate.

First-order Upwind: In this scheme the face value is approximated by the nearest neighboring cell center value depending upon the direction of the local velocity at the face.

$$(F_e \phi_e)_{FOU} = [F_e, 0] \phi_P - [-F_e, 0] \phi_E, \quad (F_w \phi_w)_{FOU} = [F_w, 0] \phi_W - [-F_w, 0] \phi_P$$

where $[a, b]$ implies the maximum of a and b .

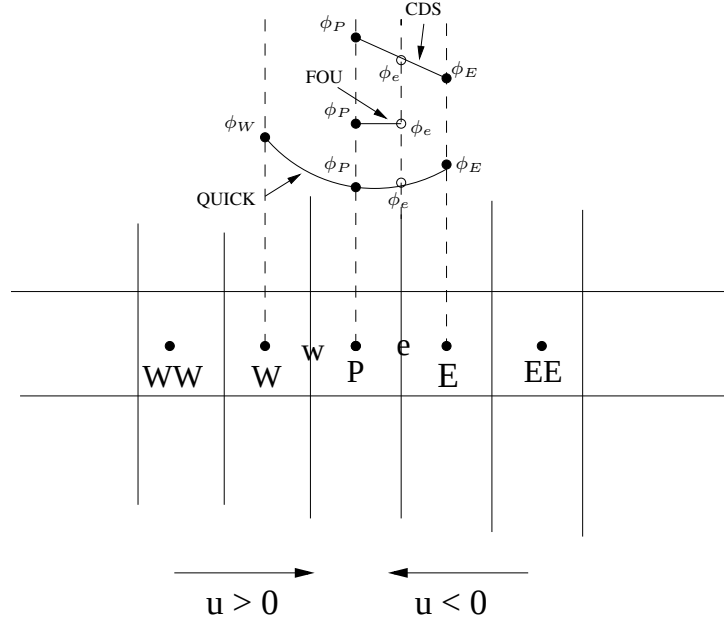


Figure 5: A typical two-dimension regular finite volume with its neighbors to demonstrate implementation of the convection scheme

Hybrid scheme: The central difference and the first-order upwind scheme can be combined to form the hybrid scheme which have the advantages of both the contributors.

$$\begin{aligned}(F_e \phi_e)_{HYB} &= \alpha (F_e \phi_e)_{CDS} + (1 - \alpha) (F_e \phi_e)_{FOU} \\ (F_w \phi_w)_{HYB} &= \alpha (F_w \phi_w)_{CDS} + (1 - \alpha) (F_w \phi_w)_{FOU}\end{aligned}$$

where $0 \leq \alpha \leq 1$, and $\alpha = 1$ corresponds to a central difference formula while for $\alpha = 0$ it becomes a purely first-order upwind scheme.

QUICK scheme: QUICK stands for “**Q**uadratic **U**pwind **I**nterpolation for **C**onvective **K**inematics”. This scheme fits a quadratic polynomial among two upstream neighbors and a downstream neighbor.

$$(F_e \phi_e)_{QUICK} = [F_e, 0](a_{UU}\phi_W + a_U\phi_P + a_D\phi_E) - [-F_e, 0](b_{UU}\phi_{EE} + b_U\phi_E + b_D\phi_P)$$

where

$$\begin{aligned}a_{UU} &= -\frac{V_E V_P}{(V_P + V_W)(V_E + 2V_P + V_W)}, \quad a_U = \frac{V_E(2V_P + V_W)}{(V_E + V_P)(V_P + V_W)} \\ a_D &= \frac{V_P(2V_P + V_W)}{(V_E + V_P)(V_E + 2V_P + V_W)}\end{aligned}$$

and

$$\begin{aligned}b_{UU} &= -\frac{V_P V_E}{(V_E + V_{EE})(V_P + 2V_E + V_{EE})}, \quad b_U = \frac{V_P(2V_E + V_{EE})}{(V_P + V_E)(V_E + V_{EE})} \\ b_D &= \frac{V_E(2V_E + V_{EE})}{(V_P + V_E)(V_P + 2V_E + V_{EE})}\end{aligned}$$

8 Immersed Boundary Method

The diffuse interface immersed boundary method is developed and implemented in order to handle multiple complex objects both in stationary and dynamic modes. In this method first an arbitrary shaped boundary is linearized - in 2D linear elements and in 3D triangular elements. The mesh on which fluid flow equations are solved are generated such that the near boundary regions receive greater focus in order to resolve finer details, i.e., grids are refined near walls. In dynamical situations only the complex boundaries move on the fixed background mesh.

8.1 Arbitrary motion of an object

Three-dimensional motion of an object can be described by three translational $\mathbf{x} = (x, y, z)$ and three rotational $\theta = (\theta_x, \theta_y, \theta_z)$ degrees-of-freedom, though it reduces to three (x, y, θ_z) in two-dimension. For forced motion of an object $\mathbf{x}$ and θ are prescribed but in case of flow-induced motion, the fluid force $\mathbf{f}$ and torque $\mathbf{T}$ tend to displace the object while the object resists them owing to its damping and stiffness properties. Moreover, external forces $\mathbf{g}$ such as weight of the object, magnetic/electric field etc. are also responsible for movement of an object immersed in a fluid. While in case of forced motion the object does not interact with the fluid, induced motion of an object due to fluid or external forces sends a non-zero feedback to the fluid leading to a two-way coupling between the fluid and object popularly known as fluid structure interaction (FSI). In such a situation prediction of the object motion becomes as important as the fluid flow solutions since they are strongly connected. Newton's law of motion can be written to adequately deduce arbitrary motion of a rigid body

$$m_s \frac{d^2 \mathbf{x}}{dt^2} + c_s \frac{d\mathbf{x}}{dt} + k_s \mathbf{x} = \mathbf{f} + \mathbf{g} \quad (38)$$

$$I_s \frac{d^2 \boldsymbol{\theta}}{dt^2} = \mathbf{T} \quad (39)$$

where m_s, c_s, k_s, I_s are mass, damping, stiffness coefficients and moment of inertia of an object.

8.2 Discrete global equations

In immersed boundary method an arbitrary shaped boundary is first linearized in order to ascertain its relation with the computing cells surrounding it. Moreover, in discrete forcing approach such linearization facilitates geometric construction required to reconstruct flow solution near the boundary, thereby enforcing boundary conditions on the body contour. However, diffuse interface method does not essentially demarcate the body surface from the fluid region as it does not aim to develop any smooth continuous reconstruction for oscillation-free solution. Instead it relies on smooth transition of a boundary through computing cells which eliminates the possible velocity and pressure discontinuity typically seen for sharp interface IBM.

This method starts with a set of universal equations that apply to all cells irrespective of its location and status. Since the method is entirely based on finite-volume formulation, numerical approximation of Eqs. 1 and 2 are written in the form one obtains after integrating over a finite

volume and introducing time discretization (excluding source terms)

$$\sum_f F_f^{n+1} = 0 \quad (40)$$

$$(1 - \phi_B) \left[\frac{u_{i,P}^{n+1} - u_{i,P}^n}{\Delta t} V_P + \aleph \left(\frac{\partial(u_i u_j)}{\partial x_j} \right) + \aleph \left(\frac{\partial p}{\partial x_i} \right) - \alpha_M \aleph \left(\frac{\partial^2 u_i}{\partial x_j \partial x_j} \right) \right] - \phi_B \frac{u_{i,P}^{n+1} - u_{i,B}}{\Delta t} V_P = 0 \quad (41)$$

where $\aleph$ is the numerical approximation, ϕ_B is the fraction of a computing cell denoted by P and having volume V_P that lies inside an object, i is the Cartesian component, $u_{i,B}$ is the i -th component of body velocity and $n + 1$ is the intended time step index. Note as $u_{i,P}^{n+1}$ is the dependent variable in Eq. 41, algebraic sum of the coefficients of it in first and last terms should be unity (here $1 - \phi_B + \phi_B = 1$). Thus, temporal approximation of the unsteady term in the momentum equation should decide form of the additional last term in Eq. 41. It can be clearly seen that in the fluid ($\phi_B = 0$) and solid ($\phi_B = 1$) regions the Navier-Stokes equations and body velocity relation ($u_{i,P}^{n+1} = u_{i,B}$) are identically satisfied. However, for the regions (read cells) through which the linearized body surface passes ($0 < \phi_B < 1$), a volume averaged effect of both the equations is in action. This step is at the heart of this method as it clearly indicates in order to avoid sharp discontinuity across a moving surface, the ever changing sharp interface between the moving object and fluid needs to be handled in a diffuse manner. Thus, the effect of no-slip condition is distributed over a computing cell and in the limit $\Delta x, \Delta y \rightarrow 0$, the discrete forcing approach is retrieved. This observation points out that required resolution in this method is higher as compared to a sharp interface method which is later shown to be non significant.

8.3 Efficient and fast calculation of volume fraction

If all the computing cells are classified as inside the body (S : solid cell), completely immersed in fluid (F : fluid cell) and containing object boundary (SF : interfacial cell), as demonstrated in Fig. 6(a), then only the interfacial cells need to be processed further for volume fraction since others will have values either 0 or 1.

Every computing cell is associated with the nearest linear element which also yields the nearest object. Although, this is only required to be computed for cells nearby an object, as those away from it already have a volume fraction (0 or 1). Use of a bounding box, shown in Fig. 6(a) (dashed line), can be adopted to eliminate superfluous calculations. This is especially true for large scale 3D computations as ratio of immersed cells to total computing cells drops significantly. However, it only saves CPU-time in serial mode and since in parallel mode different processors handle computing cells of varied status, use of a bounding box greatly disturbs the already achieved load balancing. Moreover, for moving boundary problems, unlike stationary cases, calculation of nearest element is required at each time interval as the object boundaries (read linear elements) change their position. In the present technique nearest element calculation is optimized with the aim to reduce this overhead to only a small fraction of the CPU time per time interval.

If it is assumed that rate of excursion of an object is not large then nearest element of a computing cell remains within a small neighborhood of the nearest element of the preceding cell. Let us assume set of all the computing cells is C , nearest element of an object b to a computing cell i is $Nl_b(i)$, global nearest element of i ($i \in C$) is $Nl(i)$, set of all the cells that share an edge (in 2D) or face (in 3D) with the cell i is $T(i)$ and a predefined non-zero set defining local neighborhood of a linear

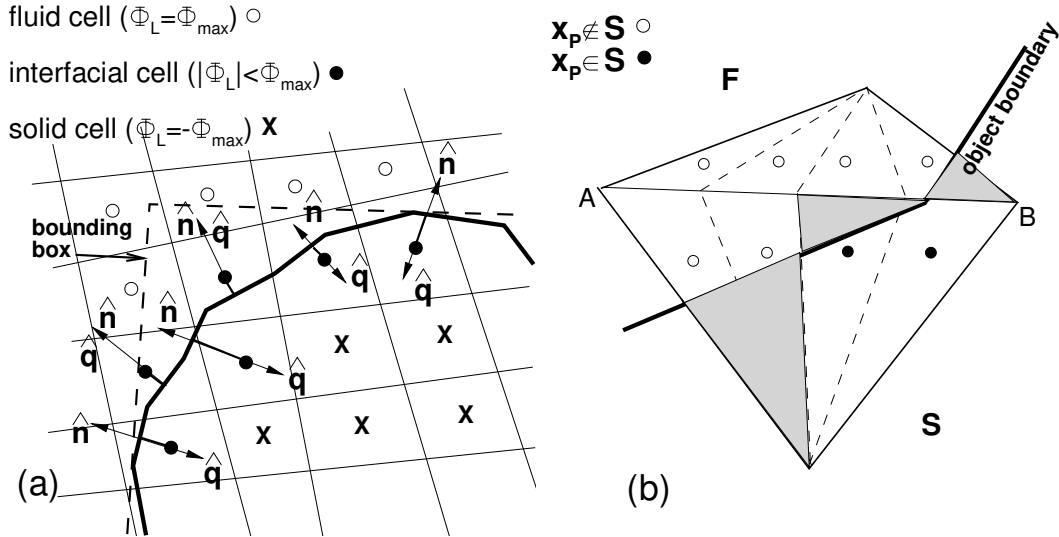


Figure 6: Geometric construction showing (a) classification of computing cells and (b) calculation of volume fraction with associated error (shaded area)

element L_b of an object b (linearized by $N_b(b)$ number of elements) be $H_b(L_b)$. Thus,

$$H_k(i) \cap H_k(j) \neq \emptyset, \quad j \in H_k(i), \quad i \in H_k(j), \quad i, j \in \{1, \dots, N_b(k)\}, \quad k = 1, \dots, b$$

where $\emptyset$ is the empty set. If L_{bP} is the nearest element of object b to a computing cell c_P then global nearest cell can be found by minimizing the distance function $|Nl_b(c_P) - c_P|$

$$Nl_b(c_P) = L_{bP}, \quad c_P \in C \implies Nl(c_P) = \min\{|Nl_k(c_P) - c_P|, k = 1, \dots, b$$

and nearest element of all the cells that share an edge (in 2D) or face (in 3D) with c_P will lie in $H_b(L_{bP})$, i.e.,

$$Nl_b(j) \in H_b(L_{bP}), \quad j \in T(c_P)$$

In case of moving objects as the relative location of the cells can change at every instant, local neighborhood of an element accelerates finding the nearest element for a computing cell. This idea is pictorially shown in Fig. 7 for three (A, B, C) computing cells. Algorithm for finding nearest element of a computing cell for a case with b objects having $N_b(i), i = 1, \dots, b$ elements proceeds as

- Create a predefined local neighborhood for each element of each object while linearizing the object(s) and load the entire data on each computing processor, $H_j(i); i = 1, N_b(j); j = 1, b$
- For the first computing cell ($i = 1$), each processor finds the nearest element of each object, $Nl_i(1), i = 1, \dots, b$, by searching among all elements and finally determines the closest one by minimizing the distance, $Nl(1) = \min\{|Nl_i(1) - 1|\}, i = 1, \dots, b$.

for computing cell $i \in C, i > 1$

- Find nearest element of object $j = 1, \dots, b, Nl_j(i)$ by searching the set $H_j(Nl_j(i - 1))$.

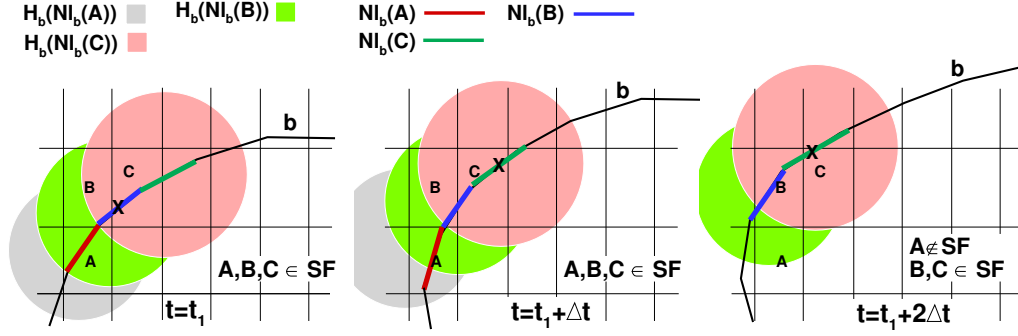


Figure 7: Relative location of three computing cells (A, B, C) and local neighborhood of body elements for a moving boundary case.

- Determine the global nearest element by $Nl(i) = \min\{|Nl_k(i) - i|\}, k = 1, \dots, b$
end for

The information of nearest element greatly reduces the task of identifying a cell. Suppose, a computing cell $i \in C$ centered at P has v_i number of vertices labeled as P_i and global nearest element $Nl(i)$ whose unit outward normal and centroid are $\hat{n}$ and Q , respectively, such that $\overline{QP}/|\overline{QP}| = \mathbf{q}$ and $\overline{QP_i}/|\overline{QP_i}| = \mathbf{q_i}$. The relative location of the vertices can be used to distribute a scalar

$$\text{if } \hat{n}(Nl(P)) \cdot \mathbf{q_i} > 0, \phi_i(P) = 1 \quad (42)$$

$$\text{if } \hat{n}(Nl(P)) \cdot \mathbf{q_i} < 0, \phi_i(P) = -1 \text{ for } 1 \leq i \leq v_i \quad (43)$$

which yields an unique integer function $\Phi_L(P)$ as

$$\Phi_L(P) = \sum_{i=1}^{v_i} \phi_i(P)$$

with clearly $\Phi_{L,max} = v_i$ and $\Phi_{L,min} = -v_i$. Thus, a computing cell $i \in C$ can be classified as

$$\text{if } \Phi_L(i) = \Phi_{L,max}(i), i \in F \text{ and } \phi_B(i) = 0$$

$$\text{if } \Phi_L(i) = \Phi_{L,min}(i), i \in S \text{ and } \phi_B(i) = 1$$

$$\text{if } |\Phi_L(i)| < \Phi_{L,max}(i), i \in SF$$

which leaves only the task to compute volume fraction for a cell i such that $i \in SF$.

A divide and conquer algorithm is adopted to accurately compute volume fraction of the interfacial cells. A host cell P having volume V_P , as shown in Fig. 6(b) is recursively divided into a finite number of triangles (2D) or prisms (3D) with centroids p . Then the unit step function is associated with each spawned cell as

$$\delta(p) = \begin{cases} 1, & p \in S, \\ 0, & p \notin S \end{cases}$$

where Eqs. 42,43 are used to ascertain relative position of a centroid p . Thus, volume occupied by the region inside an object is

$$V_S = \int_V \delta(p) dv$$

where V is the region of the host cell and dv is the volume of the spawned cells. This estimate can now be used to obtain the volume fraction

$$\phi_B(P) = \frac{V_S}{V_P} = \frac{1}{V_P} \int_V \delta(p) dv$$

where dv is elemental volume. The division of a typical cell along with error (shaded area) incurred in the process is shown in Fig. 6(b). Since the error directly depends upon the magnitude of the divided volumes, it can be reduced by employing a large number of division of the host cell. However, numerical experiments have shown that three to four levels of recursion is sufficient to bring down this error below a threshold as small as 0.04%. Overall, to reduce the cost of repeated calculations of volume fraction for moving boundary cases an additional data set containing information of local neighborhood of a linear element is created. Though, such an exercise turns out to be costly for 3D objects whose number of elements easily runs into thousands. However, as the local neighborhood is generated only at the pre-processing stage, saving in CPU time for recreating volume fraction field at every time interval clearly accelerates the calculation. Numerical overhead of this procedure has been seen to be smaller than 10% of the CPU time taken by the flow solver per time step even in most demanding cases. For two dimensional cases, owing to limited number of linear elements, this procedure has negligible share of CPU time.

8.4 Incremental motion of a rigid body

In order to accommodate arbitrary motion of a rigid body, different coefficients (see Eqs. 38,39) in the Cartesian directions need to be considered, i.e., $c_s = (c_{sx}, c_{sy}, c_{sz})$, $k_s = (k_{sx}, k_{sy}, k_{sz})$. When Eq. 38 is cast into an initial value problem one obtains a first-order system written in matrix form

$$\dot{\mathbf{q}} = \mathbf{A}\mathbf{q} + \mathbf{f} + \mathbf{g} \text{ where } \mathbf{q} = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z}]^T \text{ and}$$

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -c_{x2} & 0 & 0 & -c_{x1} & 0 & 0 \\ 0 & -c_{y2} & 0 & 0 & -c_{y1} & 0 \\ 0 & 0 & -c_{z2} & 0 & 0 & -c_{z1} \end{bmatrix}$$

with $(c_{x1}, c_{y1}, c_{z1}) = (c_{sx}, c_{sy}, c_{sz})/m_s$ and $(c_{x2}, c_{y2}, c_{z2}) = (k_{sx}, k_{sy}, k_{sz})/m_s$. Second order time-integration of the system leads to

$$\frac{1}{\Delta t}(\mathbf{q}^{n+1} - \mathbf{q}^n) = \frac{1}{2}\mathbf{A}(\mathbf{q}^{n+1} + \mathbf{q}^n) + \mathbf{f}^{n+1} + \mathbf{g}^{n+1}$$

which can be solved for $\mathbf{q}^{n+1}$ as

$$\mathbf{q}^{n+1} = (I - \frac{\Delta t}{2}\mathbf{A})^{-1}(I + \frac{\Delta t}{2}\mathbf{A})\mathbf{q}^n + (I - \frac{\Delta t}{2}\mathbf{A})^{-1}(\mathbf{f}^{n+1} + \mathbf{g}^{n+1}) \quad (44)$$

Note as $\mathbf{f}^{n+1}$ contains fluid forces, typically drag and lift coefficients, it is approximated by an explicit approach since at the start of a time step aerodynamic force coefficients are not available.

If an object rotates by an angle θ about an axis with unit vector $\hat{e}$, then the rotation matrix can be

written using the covariance of tensor equations

$$R_{ij} = \alpha \delta_{ij} + \beta e_i e_j + \gamma \epsilon_{ijk} e_k \quad (45)$$

where R_{ij} is the second rank rotation tensor, δ_{ij} is the second rank Kronecker delta (unit matrix), ϵ_{ijk} is the third rank Levi-Civita tensor and $\alpha, \beta, \gamma (\in \mathbb{R})$ are three real scalars. Now, since the rotation axis remains the same under the prescribed rotation, i.e.,

$$R_{ij} e_j = e_i$$

the contraction follows as

$$\alpha \delta_{ij} e_j + \beta e_i e_j + \gamma \epsilon_{ijk} e_j e_k = e_i$$

which owing to

$$\delta_{ij} e_j = e_i, \quad e_j e_j = 1, \quad \epsilon_{ijk} e_j e_k = 0$$

leads to

$$\alpha + \beta = 1$$

But since Eq. 45 is completely general it should hold a specific case such as $\hat{e} = (0, 0, 1)$. Thus, $R_{11} = \cos \theta = \alpha$ and $R_{12} = -\sin \theta = \gamma$ which yields the scalars as $\alpha = \cos \theta$, $\beta = 1 - \cos \theta$, $\gamma = -\sin \theta$. Thus the rotational matrix becomes

$$R_{ij} = \cos \theta \delta_{ij} + (1 - \cos \theta) e_i e_j - \sin \theta \epsilon_{ijk} e_k \quad (46)$$

which in matrix form is given by

$$\begin{bmatrix} (1 - \cos \theta) e_x^2 + \cos \theta & (1 - \cos \theta) e_x e_y - e_z \sin \theta & (1 - \cos \theta) e_x e_z + e_y \sin \theta \\ (1 - \cos \theta) e_x e_y + e_z \sin \theta & (1 - \cos \theta) e_y^2 + \cos \theta & (1 - \cos \theta) e_y e_z - e_x \sin \theta \\ (1 - \cos \theta) e_x e_z - e_y \sin \theta & (1 - \cos \theta) e_y e_z + e_x \sin \theta & (1 - \cos \theta) e_z^2 + \cos \theta \end{bmatrix}$$

Note that $\hat{e}$ is determined by the direction of net fluid torque $\mathbf{T}/|\mathbf{T}|$ and θ is the solution of the initial value problem $\dot{\theta} = \omega$, $\dot{\omega} = |\mathbf{T}|/I_s$ with $\theta(t=0) = \theta_0, \omega(t=0) = \omega_0$. By using the translational ($\mathbf{q}^{n+1}$) and angular R_{ij}, ω^{n+1} displacements and velocities the moving objects can be traced accurately. Specific entries of $\mathbf{A}, \mathbf{f}, \mathbf{g}$ and R_{ij} will be mentioned for different test cases.

8.5 Fluid flow solution

The governing equations are solved using a cell-centered finite volume method with the variables defined in a non-staggered fashion. The fully discrete form of Eqs. 1-2 are

$$\sum_f F_f^{n+1} = 0 \quad (47)$$

$$\begin{aligned} (1 - \phi_B) \left[\frac{u_{i,P}^{n+1} - u_{i,P}^n}{\Delta t} V_P + \sum_{l=-p}^q \alpha_l \sum_f F_f^{n+l} u_{i,f}^{n+l} + \frac{1}{2} \left(\sum_f p_f^{n+1} S_{f,i} + \sum_f p_f^n S_{f,i} \right) \right. \\ \left. - \frac{1}{2Re} \left(\sum_f F_{dfu_i}^{n+1} + \sum_f F_{dfu_i}^n \right) \right] + \phi_B \frac{u_{i,P}^{n+1} - u_{i,B}^n}{\Delta t} V_P = 0 \end{aligned} \quad (48)$$

Note $q = 1$ or 0 refers to fully-implicit or semi-implicit time marching, respectively, with $p > 0$ facilitates higher-order temporal approximation for the non-linear convective term. All the linear terms are handled with standard second-order Crank-Nicolson scheme to add numerical stability to the overall time marching algorithm. Since the latest pressure p^{n+1} that correctly couples the velocity field is not known, the above equation is first solved without it for a provisional velocity field u_i^*

$$(1 - \phi_B) \left[\frac{u_{i,P}^* - u_{i,P}^n}{\Delta t} V_P + \sum_{l=-p}^0 \alpha_l \sum_f F_f^{n+l} u_{i,f}^{n+l} + \alpha_1 \sum_f F_f^* u_{i,f}^* \right. \\ \left. - \frac{1}{2Re} \left(\sum_f F_{dfu_i}^* + \sum_f F_{dfu_i}^n \right) \right] + \phi_B \frac{u_{i,P}^* - u_{i,B}}{\Delta t} V_P = 0$$

To achieve a stronger coupling between the non-staggered velocity and pressure, a local pressure gradient is then added with the mass flux obtained from the provisional velocity

$$F_f = F_f^* - \Delta t (\nabla p)_f \cdot \mathbf{S}_f$$

which when inserted in Eq. 47, a Poisson equation for the pressure (p^{n+1}) is obtained as

$$\sum_f (\nabla p)_f \cdot \mathbf{S}_f = \frac{1}{\Delta t} \sum_f F_f^* \quad (49)$$

Such an elliptic equation for the pressure (or a pressure like variable) is well known for incompressible flow algorithms. Solution of this equation, p^{n+1} , can be inserted back into Eq. 48 to finally obtain the physically correct velocity field, u_i^{n+1} . A detailed discussion on the treatment and solution methodology can be found in [].

8.6 Force and moment coefficients

Since this method originally does not demarcate the regions between a solid and fluid “sharply”, calculation of aerodynamic coefficients proceeds with volume integral constructed over a specific object. However, as the stress tensor $\sigma_{ij} = -p\delta_{ij} + \mu(\partial u_i/\partial x_j + \partial u_j/\partial x_i) = -p\delta_{ij} + 2\mu S_{ij}$ (S_{ij} symmetric strain rate tensor) acts on a differential surface element of an object, it needs to be converted to a volume integral. If S_b is the bounding surface of an object b of volume V_b then the normalized force can be obtained by integrating the surface force ($\bar{\sigma} \cdot \mathbf{n}$) over an elemental area dS with outward unit normal $\mathbf{n}$.

$$\begin{aligned} \mathbf{F}_b &= \int_{S_b} \sigma_{ij} n_j dS \\ &= \int_{S_b} \left[-p\delta_{ij} + \frac{1}{Re} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \right] n_j dS \\ &= \int_{V_b} \left[-\frac{\partial p}{\partial x_j} \delta_{ij} + \frac{1}{Re} \frac{\partial^2 u_i}{\partial x_j \partial x_j} + \frac{1}{Re} \frac{\partial}{\partial x_j} \left(\frac{\partial u_j}{\partial x_i} \right) \right] dV \\ &= \int_{V_b} \left(-\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u} \right) dV \end{aligned} \quad \left(\text{as } \frac{\partial u_j}{\partial x_j} = 0 \right)$$

$$= \sum_N \phi_B \left(\sum_f -p_f \mathbf{S}_f + \frac{1}{Re} \sum_f \mathbf{F}_{df} \right)$$

where N refers to all computing cells. On the other hand, normalized moment can be computed by taking cross-product of the surface force, $\mathbf{r} \times (\bar{\boldsymbol{\sigma}} \cdot \mathbf{n})$

$$\begin{aligned} \mathbf{M}_b &= \int_{S_b} \epsilon_{ijk} r_j \sigma_{kl} n_l dS \\ &= \int_{S_b} [-\epsilon_{ijk} r_j p \delta_{kl} n_l + \frac{2}{Re} \epsilon_{ijk} r_j S_{kl} n_l] dS \\ &= \int_{V_b} [-\epsilon_{ijk} \delta_{jk} p - \epsilon_{ijk} r_j \frac{\partial p}{\partial x_k} + \frac{2}{Re} \epsilon_{ijk} \delta_{jl} S_{kl} + \frac{1}{Re} \epsilon_{ijk} r_j \frac{\partial}{\partial x_l} (\frac{\partial u_l}{\partial x_k}) \\ &\quad + \frac{1}{Re} \epsilon_{ijk} r_j \frac{\partial^2 u_k}{\partial x_l \partial x_l}] dV \\ &= \int_{V_b} [-\epsilon_{ijk} r_j \frac{\partial p}{\partial x_k} + \frac{2}{Re} \epsilon_{ijk} S_{jk} + \frac{1}{Re} \epsilon_{ijk} r_j \frac{\partial^2 u_k}{\partial x_l \partial x_l}] dV \\ &\quad \text{(as } \epsilon_{ijj} = 0, \frac{\partial u_l}{\partial x_l} = 0) \\ &= \int_{V_b} \mathbf{r} \times (-\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u}) dV \\ &\quad \text{(as } \epsilon_{ijk} S_{jk} = \epsilon_{ikj} S_{kj} = \epsilon_{ikj} S_{jk} = -\epsilon_{ijk} S_{jk}) \\ &= \sum_N \phi_B [\sum_f -p_f (\mathbf{r} \times \mathbf{S}_f) + \frac{1}{Re} \sum_f \mathbf{r} \times \mathbf{F}_{df}] \end{aligned}$$

9 Parallelization

The entire code is parallelized using both shared and distributed memory forms. In shared memory, popularly known as OPENMP, the entire data set is shared among commuting entities known as threads. While every threads perform the task given to it independently, the data is shared among them. On the other hand, in distributed memory system the computing domain is subdivided into a number of sub-regions, known as domain decomposition, and each of these sub-region is assigned to a processor which does not have any knowledge of the other. Distributed parallel code is written by using standard calls of MPI routines. Evidently, owing to memory requirement large size of problems always favor distributed memory based parallelization as the entire memory is split into a number of chunks among participating processors, unlike shared memory calculations where all the threads share a single large memory.

Sample 2D and 3D grids distributed among a number of processors are shown in Fig. 8 alongside linearized objects. Usually processors are denoted as “rank” which starts from 0. In the figure note the order in which the processors are distributed in coordinate directions with r, θ, z in cylindrical coordinate system have the same order as x, y, z in Cartesian coordinate system. A MPI code starts with the initialization

call **MPI_INIT(ierr)**

followed by creating an environment “MPI_COMM_WORLD” with “np” number of processors

call `MPI_COMM_SIZE(MPI_COMM_WORLD,np,ierr)`

where the processors are defined (here by “rank”)

call `MPI_COMM_RANK(MPI_COMM_WORLD,rank,ierr)`

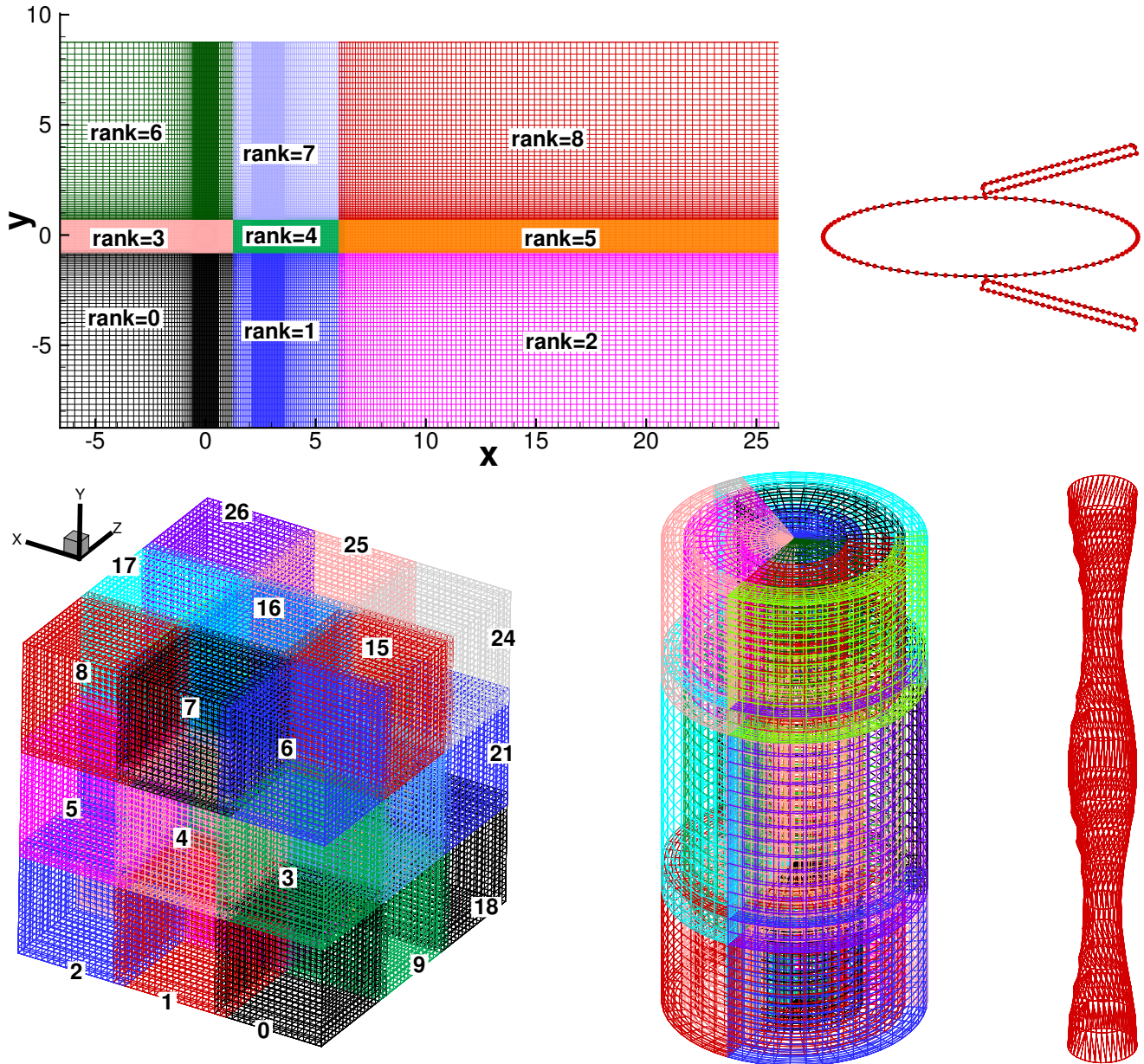


Figure 8: Sample 2D and 3D grids decomposed in 9 (2D) and 27 (3D) processors shown in different colors alongside linearized objects.

Since the entire data set is independently defined in processors, a mapping is used to relate a global cell counter to its local number in a processor. Subsequently all the processors are assigned tasks keeping in mind that the load on each of them is as identical as possible. Along a interprocessor boundary data communication is necessary as each processor only deals with its own data and has no

knowledge about its neighbors. This scenario is pictorially depicted in Fig. 9 where data dependency is shown for a 2D decomposition. There are a number of categorically different communication calls:

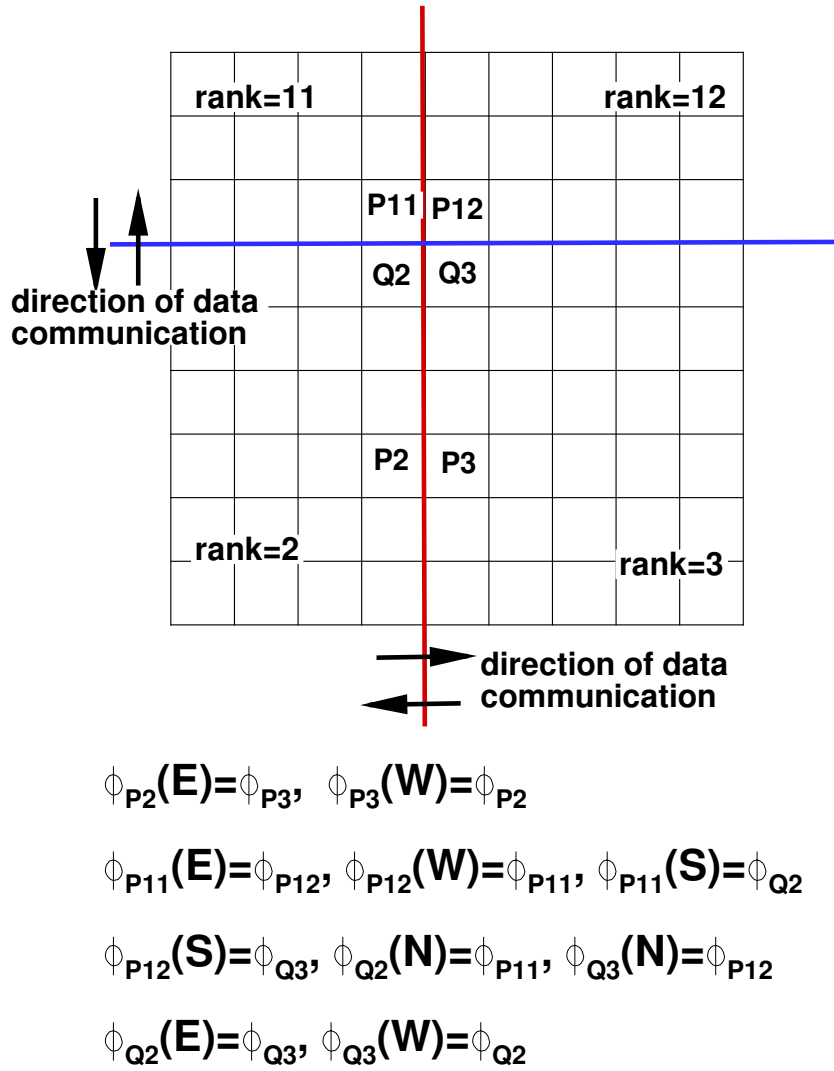


Figure 9: Data dependency and data communication in a sample 2D grid.

A point-to-point communication occurs between two processors (sender and receiver) which includes the couple

call `MPI_SEND( $\phi$ ,size,DATATYPE,receiver,flag,MPI_COMM_WORLD,ierr)`
call `MPI_RECV( $\phi$ ,size,DATATYPE,sender,flag,MPI_COMM_WORLD,status,ierr)`

A point-to-all communication occurs when a single processor (ROOT) communicates with rest of the processors

call `MPI_BCAST( $\phi$ ,size,DATATYPE,ROOT,MPI_COMM_WORLD,ierr)`

A collective communication occurs when a data (ϕ) is processed (ϕ_r) from all processor to a single processor (ROOT)

call **MPI_REDUCE**($\phi, \phi_r, \text{size}, \text{DATATYPE}, \text{MPI_OPERATION}, \text{ROOT}, \text{MPI_COMM_WORLD}, \text{ierr}$)

or to all processors

call **MPI_ALLREDUCE**($\phi, \phi_r, \text{size}, \text{DATATYPE}, \text{MPI_OPERATION}, \text{MPI_COMM_WORLD}, \text{ierr}$)

where “MPI_OPERATION” refers to operation nature and among others the most useful and important ones are **MPI_SUM**, **MPI_MAX**, **MPI_MIN**, **MPI_PROD**, **MPI_MAXLOC**, **MPI_MINLOC** for calculating sum, maximum, minimum, product, location of maximum, location of minimum, respectively. Likewise based on the nature of communication/exercise among/between processors suitable MPI libraries are used. Note during the flow of instructions synchronization among the processors is important as it is one of the objective of a parallel algorithm is to have loads of processors as identical as possible for better scalability and speed-up. Thus, uneven loads on processors are achieved by forcing a synchronization using

call **MPI_BARRIER**(**MPI_COMM_WORLD**, **ierr**)

Notice while in 2D data communication involves only exchange of 1D lines, it is 2D surface data that changes hand in 3D and thus the decomposition type does have a role in deciding the amount of communication required. At the completion of computations the MPI parallel environment is terminated by

call **MPI_FINALIZE**(**ierr**)

The input-output (in short IO) exercise in parallel environment is different than the serial mode. Note two processors can simultaneously read from a file, but can not write as they may have different values stored in their local memory address. Thus, while reading data, all the processors can read simultaneously from the same file. However, parallel write is achieved by writing each sub domain data into a different file simultaneously. Though it is possible to write data from different processor from different location of a file, in the current implementation all processors write in a different file which are suitably consolidated at the post-processing stage.

The shared memory based parallelization operates is a mode which is popularly known as the “fork-and-join” type. In this approach the code always runs on a single computing machines with one memory unit shared by a number of cores. It is the OMP environment that forks the computations into a number of **threads** and when not needed joins them into one and single (serial) processor environment is retrieved. The number of threads to be used is fed through the commands at the time of execution. The part of code to be parallelized among threads is written inside what are known as standard **OPENMP directives**

!\$omp parallel

.

!\$omp end parallel

The datatype can be declared with this directives for the part to be parallelized as

!\$omp parallel default(list of data which is defined in default) **shared**(list of data to be shared by the threads) **private**(list of data private to a thread)

.
.
.

!\$omp end parallel

Distribution of tasks among threads can be achieved by explicitly assigning threads their share of work or by simply using OMP directives to split a loop

!\$omp do reduction(operator:list of data to be operated)

.
.
.

!\$omp end do

Note the “reduction” clause is optional and among others a few important operators are **+**, **-**, *****, **max**, **min** for addition, subtraction, product, maximum and minimum, respectively. This clause can be thought of as similar to the “ALLREDUCE” library of MPI. Like in MPI synchronization among threads can be enforced by the directive

!\$omp barrier

10 Parallel preconditioner